

Desarrollo de una aplicación móvil para la gestión digital de pedidos en el restaurante “La Choza” utilizando .NET MAUI y arquitectura MVVM

Aplicación Choza POS para optimización de procesos en restaurantes

Development of a Mobile Application for Digital Order Management in the “La Choza” Restaurant Using .NET MAUI and MVVM Architecture

Choza POS Application for Restaurant Process Optimization

Nombres de Autores de Primera Institución

Alexander Fernando Tupiza Carrera, Maycol Ronald
Moreno Suquilanda, Norma Katherine Caiza Lasso
Universidad Israel Quito, Ecuador

Resumen — El presente trabajo describe el desarrollo de una aplicación móvil denominada *Choza POS*, orientada a digitalizar y optimizar la gestión de pedidos en el restaurante “La Choza”. El sistema reemplaza el proceso manual basado en comandas en papel, permitiendo registrar, actualizar y monitorear pedidos en tiempo real desde dispositivos móviles. La solución está desarrollada utilizando el framework .NET MAUI bajo el patrón de arquitectura MVVM, garantizando una separación clara entre la lógica de negocio y la interfaz de usuario. Además, la aplicación se integra con un backend mediante servicios REST con autenticación basada en tokens JWT, asegurando la protección de los datos. Como resultado, el sistema mejora la comunicación entre el personal del restaurante, reduce errores operativos y optimiza los tiempos de atención, contribuyendo a una gestión más eficiente

Palabras Clave - Aplicación móvil, .NET MAUI, MVVM, gestión de pedidos, REST API, restaurante.

Abstract — This paper presents the development of a mobile application called *Choza POS*, designed to digitalize and optimize order management in the “La Choza” restaurant. The system replaces manual processes based on paper orders, allowing real-time registration, updating, and monitoring of orders from mobile devices. The solution is developed using the .NET MAUI framework following the MVVM architecture pattern, ensuring a clear separation between business logic and user interface. Additionally, the application integrates with a backend through REST services with JWT-based authentication, ensuring data security. As a result, the system improves communication among

restaurant staff, reduces operational errors, and optimizes service time, contributing to more efficient management.

Keywords - Mobile application, .NET MAUI, MVVM, order management, REST API, restaurant.

I. INTRODUCCIÓN

En la actualidad, la digitalización de procesos en el sector gastronómico se ha convertido en una necesidad para mejorar la eficiencia operativa y la calidad del servicio al cliente. Muchos restaurantes aún utilizan métodos tradicionales basados en papel para la toma de pedidos, lo que puede generar errores, retrasos y desorganización.

El restaurante “La Choza” presenta esta problemática en su gestión diaria, afectando la comunicación entre meseros, cocina y administración. Por ello, se propone el desarrollo de una aplicación móvil que permita optimizar el flujo de información mediante una solución tecnológica moderna.

El objetivo principal de este proyecto es desarrollar una aplicación móvil que permita gestionar pedidos, mesas y pagos de manera eficiente, reduciendo errores y mejorando los tiempos de atención.

II. ANÁLISIS E IMPLEMENTACIÓN DE REQUERIMIENTOS

A. Diseño de interfaz intuitiva

La aplicación Chozapos ha sido diseñada priorizando la usabilidad mediante una interfaz clara, organizada e intuitiva. El sistema permite al usuario navegar de manera eficiente entre las diferentes funcionalidades, tales como la gestión de mesas, registro de pedidos y procesamiento de pagos, reduciendo la curva de aprendizaje y optimizando los tiempos de operación.

B. Interacción eficiente del usuario

Se han implementado mecanismos que facilitan la interacción del usuario, incluyendo accesos directos, indicadores visuales de estado (como codificación por colores para pedidos en proceso, listos o pendientes) y un sistema de notificaciones en tiempo real. Estas funcionalidades contribuyen a minimizar errores y mejorar la eficiencia durante la operación diaria.

C. Accesibilidad en dispositivos móviles

La aplicación se encuentra optimizada para dispositivos Android, garantizando un rendimiento fluido en entornos de trabajo dinámicos. Su diseño responsivo permite una experiencia de usuario consistente, facilitando su uso por parte del personal del restaurante en cualquier momento y ubicación.

D. Organización mediante arquitectura MVVM

La implementación del patrón arquitectónico Model-View-ViewModel (MVVM) permite una adecuada separación de responsabilidades dentro del sistema. Esto contribuye a mejorar la mantenibilidad, escalabilidad y evolución de la aplicación, impactando positivamente en la calidad del software y en la experiencia del usuario final.

III. DISEÑO PRELIMINAR Y MODELO DE DATOS

El desarrollo de la aplicación móvil Chozapos se realizó utilizando el framework .NET MAUI, lo que permite la creación de aplicaciones multiplataforma para dispositivos móviles. Para la estructura del sistema se implementó el patrón arquitectónico Model-View-ViewModel (MVVM), el cual facilita la separación entre la lógica de negocio, la interfaz de usuario y la gestión de datos.

La aplicación se conecta a un backend mediante servicios REST, permitiendo la comunicación en tiempo real entre los distintos módulos del sistema y garantizando la persistencia de la información en la base de datos PostgreSQL.

El modelo de datos del sistema se presenta en la Figura 1, donde se identifican las principales entidades, tales como Usuario, Pedido, Producto, Cliente y Pago, así como sus respectivas relaciones, permitiendo estructurar la información de manera organizada y eficiente.

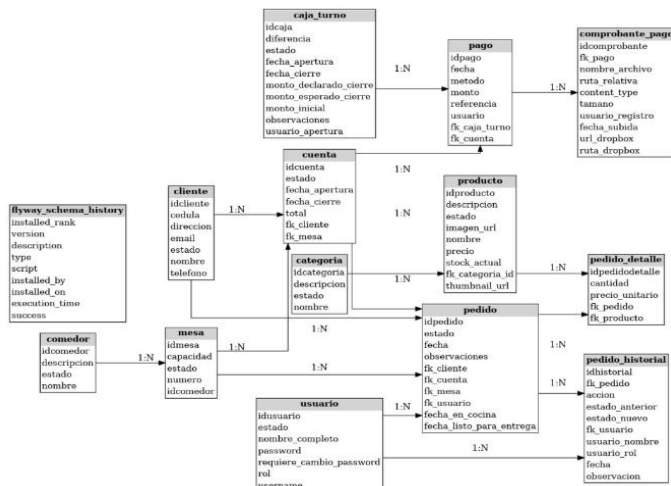


Figura 1. Modelo entidad-relación del sistema Chozapos.

IV. ARQUITECTURA DEL SISTEMA

La arquitectura del sistema Chozapos sigue un modelo cliente-servidor, en el cual la aplicación móvil desarrollada en .NET MAUI funciona como cliente y se comunica con un backend implementado en Spring Boot utilizando Java 17.

El cliente móvil, desarrollado en Visual Studio, se encarga de la interacción con el usuario, permitiendo la gestión de mesas, pedidos, pagos y visualización de información en tiempo real.

El backend expone una API REST que procesa la lógica de negocio y maneja las operaciones de acceso a datos, utilizando PostgreSQL como sistema gestor de base de datos para garantizar la persistencia y consistencia de la información.

La comunicación entre la aplicación móvil y el servidor se realiza mediante protocolos HTTP utilizando JSON como formato de intercambio de datos, asegurando interoperabilidad entre los componentes del sistema.

Adicionalmente, se implementa comunicación en tiempo real mediante WebSocket con protocolo STOMP, lo que permite la sincronización inmediata de eventos como cambios de estado de pedidos entre cocina, camareros y caja.

El sistema también integra un servicio externo a través de la API de Dropbox, utilizado para almacenar comprobantes de pago capturados desde el dispositivo móvil.

La Figura 2 presenta la arquitectura general del sistema.

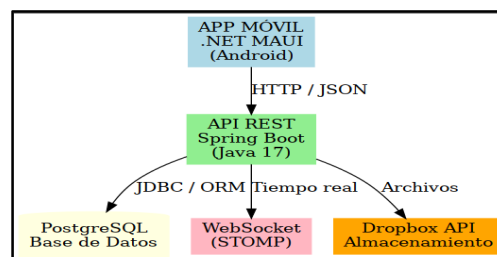


Figura 2. Arquitectura del sistema Chozapos.

GIT HUB:

<https://github.com/maycolmoreno/ChozMaui.git>

METODOLOGIA MOBIL-D

METODOLOGÍA DE DESARROLLO

Para el desarrollo de la aplicación móvil ChozPOS se utilizó la metodología ágil **Mobile-D (Mobile Development)**, la cual está orientada específicamente al desarrollo de aplicaciones móviles en equipos pequeños, permitiendo ciclos de desarrollo rápidos, iterativos e incrementales.

Esta metodología fue seleccionada debido a su enfoque en la entrega continua de funcionalidades, adaptación a cambios y validación temprana del sistema, lo cual resulta ideal para entornos dinámicos como el sector gastronómico.

Mobile-D se estructura en cinco fases principales: exploración, inicialización, producción, estabilización y pruebas finales. A continuación, se describe su aplicación en el desarrollo del sistema ChozPOS:

Fase de Exploración

En esta fase se realizó el levantamiento de información del restaurante “La Choz”, identificando los principales problemas en la gestión manual de pedidos, tales como errores en comandas, retrasos en la atención y falta de comunicación entre áreas.

Se definieron los requerimientos funcionales principales del sistema, entre los cuales se incluyen:

- Gestión de mesas
- Registro de pedidos
- Control de estados (en cocina, listo, entregado)
- Procesamiento de pagos
- Generación de comprobantes

2. Fase de Inicialización

Durante esta fase se establecieron las bases tecnológicas del sistema, definiendo la arquitectura y herramientas a utilizar.

Se seleccionaron las siguientes tecnologías:

- **.NET MAUI** para el desarrollo de la aplicación móvil
- **Arquitectura MVVM** para la organización del código
- **Spring Boot** para el backend
- **PostgreSQL** como base de datos
- **Servicios REST y WebSocket** para la comunicación

Además, se diseñaron los primeros prototipos de interfaz de usuario y el modelo de datos (entidad-relación).

3. Fase de Producción

En esta fase se desarrollaron las funcionalidades del sistema de manera iterativa, implementando módulos funcionales que fueron integrándose progresivamente.

Entre las funcionalidades implementadas se encuentran:

- Módulo de gestión de mesas
- Módulo de pedidos
- Módulo de pagos
- Visualización de estados en tiempo real

- Integración con API REST
- Subida de comprobantes mediante Dropbox

Cada funcionalidad fue desarrollada y validada de forma incremental, permitiendo detectar errores tempranamente y realizar mejoras continuas.

4. Fase de Estabilización

En esta etapa se realizó la integración completa del sistema, asegurando la correcta comunicación entre la aplicación móvil, el backend y la base de datos.

Se llevaron a cabo:

- Corrección de errores
- Optimización del rendimiento
- Validación de la arquitectura cliente-servidor
- Pruebas de sincronización mediante WebSocket

5. Fase de Pruebas y Ajustes Finales

Finalmente, se ejecutaron pruebas funcionales y técnicas para validar el correcto funcionamiento del sistema.

Se realizaron:

- Pruebas de flujo completo de pedidos
- Pruebas de API mediante Postman
- Validación de comunicación HTTP y JSON
- Pruebas en tiempo real
- Pruebas de carga y tolerancia a fallos

Los resultados obtenidos confirmaron la estabilidad, eficiencia y correcto desempeño de la aplicación ChozPOS en un entorno controlado.

TECNOLOGIAS APLICADAS

El desarrollo de la aplicación ChozPOS se realizó utilizando diversas tecnologías modernas orientadas a aplicaciones móviles y sistemas distribuidos. Para el frontend se empleó **.NET MAUI**, permitiendo la creación de una aplicación multiplataforma con lenguaje C# y estructurada bajo el patrón de arquitectura MVVM, lo que facilita la separación entre la lógica de negocio y la interfaz de usuario. En el backend se utilizó **Spring Boot** con Java 17, encargado de gestionar la lógica del sistema y exponer una API REST para la comunicación con el cliente mediante el protocolo HTTP y formato JSON. Para la persistencia de datos se implementó **PostgreSQL** como sistema gestor de base de datos relacional. Además, se incorporó **WebSocket** con protocolo STOMP para la actualización en tiempo real de los pedidos, autenticación mediante **JWT** para garantizar la seguridad, y la integración con la API de **Dropbox** para el almacenamiento de comprobantes de pago, conformando así una arquitectura robusta, escalable y segura.

Technologies Used



.NET MAUI



Spring Boot



PostgreSQL



JWT

V. FUNCIONALIDADES IMPLEMENTADAS

La **Figura 3** muestra una pantalla de una app de pedidos en el momento de cobrar. Se ve el detalle del pedido con “1x Alitas bbq” y el total de \$7.00 que aparece tanto como consumo y saldo a cobrar. Más abajo aparece la sección de método de pago con varias opciones como efectivo, tarjeta, transferencia y otro, donde transferencia está seleccionada. Debido a eso, se habilita una parte para subir el comprobante, con botones para tomar foto, elegir desde la galería o subir la imagen. Finalmente, en la parte inferior hay un botón verde que dice “COBRAR Y SUBIR COMPROBANTE” para terminar la operación.

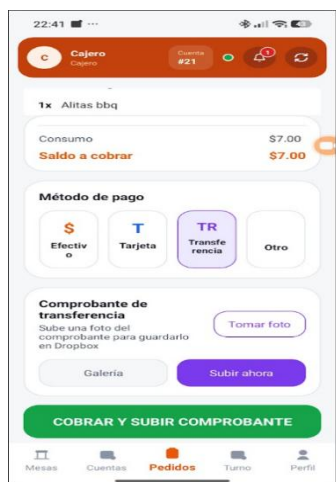


Figura 3. Pantalla de cobro con comprobante de pago

La **Figura 4** muestra una pantalla de la app donde se puede ver el estado de un pedido. Aparece una sección de observaciones con el texto “para llevar” y, más abajo, el estado actualizado como “ENTREGADO”. También se visualiza un historial con los pasos del pedido, como creado, enviado a cocina, marcado como listo y entregado al cliente, cada uno con su hora. Finalmente, en la parte inferior hay un botón “Ir a pagar” para continuar con el proceso.



Figura 4. Pantalla de historial y estado del pedido

La **Figura 5** muestra una pantalla donde se visualiza el detalle completo del pedido antes de ser entregado. Se observa la información del cliente y del cajero, junto con la lista de productos, como “Hamburguesas” y “Alitas bbq”, cada uno con su precio. También se muestra el consumo total de \$7.00. Más abajo aparece una sección de observaciones con el texto “para llevar”. Finalmente, en la parte inferior hay un botón verde con el texto “ENTREGAR AL CLIENTE”, que permite completar la entrega del pedido.

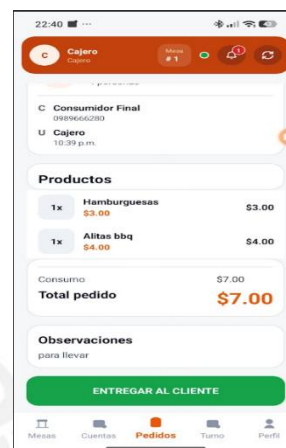


Figura 5. Pantalla de detalle del pedido y entrega

La **Figura 6** muestra una pantalla de la app donde se visualizan los pedidos que ya están listos. En la parte superior aparecen contadores indicando cuántos pedidos están en cocina, listos, del día y cancelados. La pestaña “Listos” está seleccionada, mostrando un pedido de la Mesa 1 con un total de \$7.00 y estado “LISTO”. También se puede ver un campo de búsqueda para filtrar pedidos. Finalmente, cada pedido tiene dos botones: “Ver pedido” para revisar el detalle y “Entregar” para completar la entrega.



Figura 6. Pantalla de pedidos listos para entrega

La **Figura 7** muestra una pantalla de la app donde se visualizan las mesas disponibles en el local. En la parte superior aparece un mensaje indicando que se puede tocar una mesa para crear o continuar un pedido. Debajo se observa la sección “SALA 1” con tres mesas: la mesa 1 está en estado “EN COCINA”, mientras que las mesas 2 y 3 están “LIBRES”, cada una con su capacidad de personas. También se incluye una pequeña leyenda con colores para identificar los estados de las mesas. En la parte inferior se encuentra el menú de navegación, donde la sección “Mesas” está seleccionada.

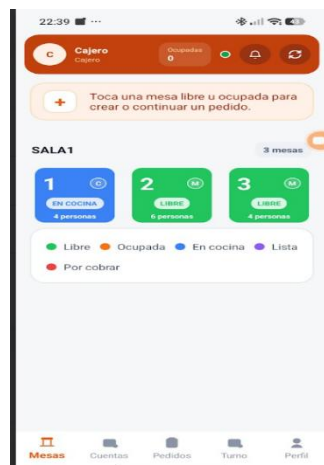


Figura 7. Pantalla de gestión de mesas

La **Figura 8** muestra una pantalla de la app donde se está creando o editando un pedido para la mesa 1. Se observa el listado de productos, como “Hamburguesas” y “Alitas bbq”, cada uno con su precio y controles para aumentar o disminuir la cantidad o eliminar el producto. También aparece el cliente asignado como “Consumidor Final” y un espacio para agregar observaciones, donde se escribe “para llevar”. En la parte inferior se muestra el total del pedido de \$7.00 y un botón principal “ENVIAR A COCINA”, junto con opciones secundarias para guardar o cancelar el pedido.

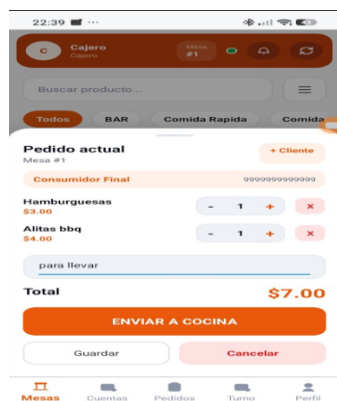
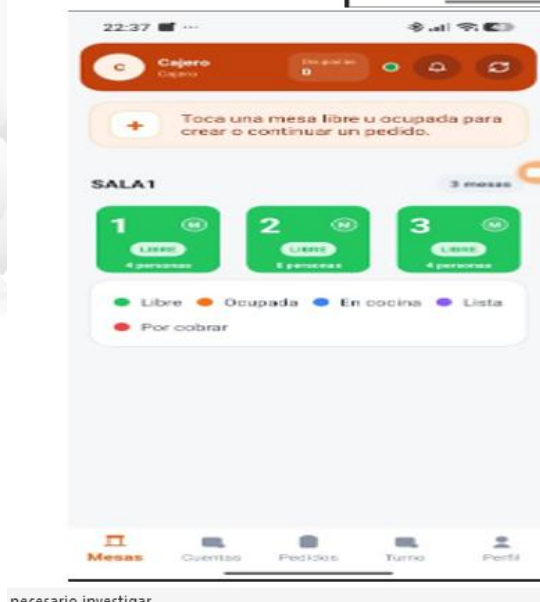
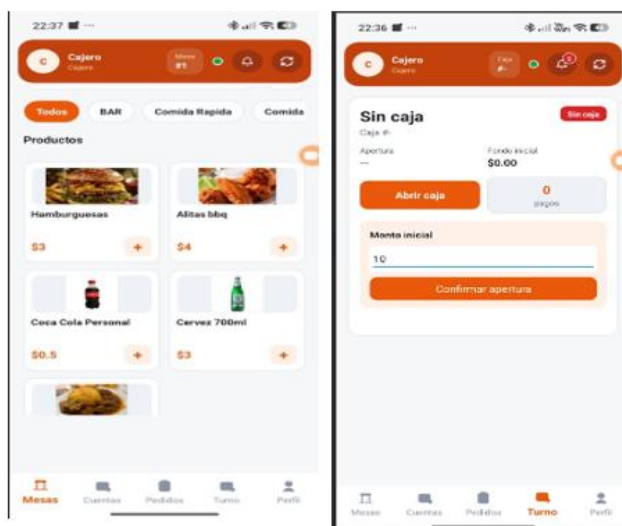


Figura 8. Pantalla de creación y gestión del pedido

La **Figura 9** muestra el proceso inicial para crear un pedido dentro de la app. Primero se visualizan las mesas disponibles en la sección “SALA 1”, donde se puede elegir una mesa libre. Al seleccionar una mesa, se despliega una ventana con la información de la mesa, como su número, capacidad y estado, junto con la opción de iniciar o continuar un pedido. Luego, se presenta la pantalla de productos, donde el usuario puede ver diferentes artículos como hamburguesas, alitas y bebidas, cada uno con su precio y un botón para agregarlos al pedido. Este conjunto de pantallas representa el flujo desde la selección de mesa hasta la elección de productos.

Figura 9. Flujo de selección de mesa y productos



VI. SERVICIOS WEB Y BASE DE DATOS

El sistema ChozaPOS implementa una arquitectura basada en servicios web, utilizando una API REST desarrollada con Spring Boot sobre Java 17 como capa de backend.

Esta API permite gestionar las operaciones del sistema, tales como autenticación de usuarios, administración de pedidos, gestión de mesas, procesamiento de pagos y consulta de información, mediante el uso de endpoints accesibles a través del protocolo HTTP y el intercambio de datos en formato JSON.

La aplicación móvil desarrollada en .NET MAUI consume estos servicios web utilizando peticiones HTTP, garantizando la comunicación entre el cliente y el servidor de manera eficiente y escalable.

Para la persistencia de la información, el sistema utiliza PostgreSQL como sistema gestor de base de datos relacional. Este componente permite almacenar de forma estructurada entidades como usuarios, clientes, pedidos, productos, cuentas y pagos, asegurando la integridad y consistencia de los datos.

La integración entre el backend y la base de datos se realiza mediante el uso de tecnologías de acceso a datos (ORM), permitiendo ejecutar operaciones de inserción, actualización, consulta y eliminación de registros.

Esta arquitectura basada en servicios web permite:

Centralizar la lógica de negocio en el servidor

Facilitar la escalabilidad del sistema

Garantizar la seguridad mediante autenticación con JWT

Permitir la interoperabilidad entre diferentes clientes

En la Figura 10 se muestra la interacción entre la aplicación móvil, la API REST y la base de datos PostgreSQL.

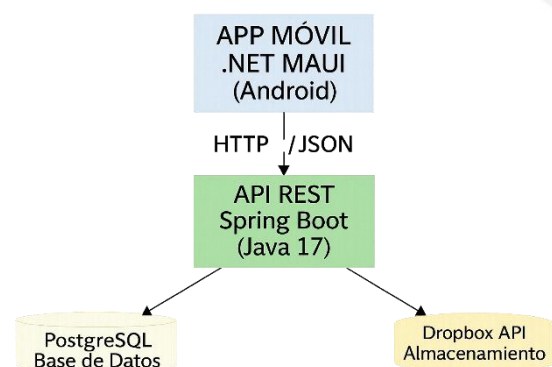


Figura 10. Servicios Web y Base de Datos.

VII. PRUEBAS DE APLICACIÓN

Se realizaron pruebas internas en un entorno controlado para validar el funcionamiento del sistema ChozaPOS. En las pruebas funcionales se verificó el flujo completo, desde el registro de pedidos hasta su procesamiento y cierre, confirmando la correcta interacción entre la aplicación móvil, la API REST y la base de datos, como se muestra en la Figura 11.

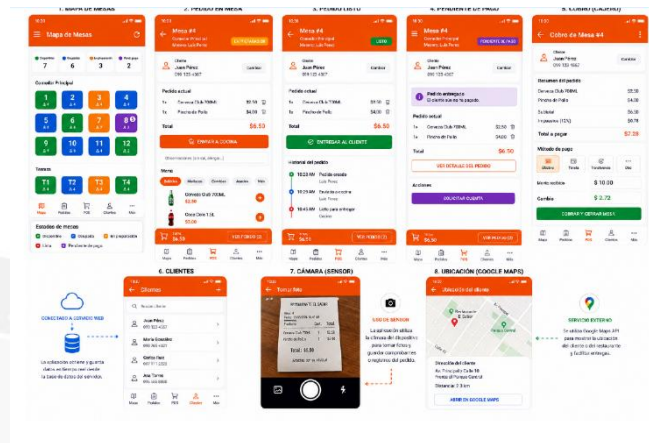


Figura 11. Funcionalidad

Asimismo, se ejecutaron pruebas de servicios web mediante Postman y Swagger, validando endpoints como autenticación, pedidos y pagos. Se comprobó la correcta comunicación HTTP y la respuesta en formato JSON, evidenciando el correcto funcionamiento del backend, tal como se observa en la Figura 12.



Adicionalmente, se evaluó la conectividad y sincronización en tiempo real, verificando la actualización de estados mediante WebSocket y el consumo de servicios desde la aplicación móvil, como se muestra en la Figura 13.

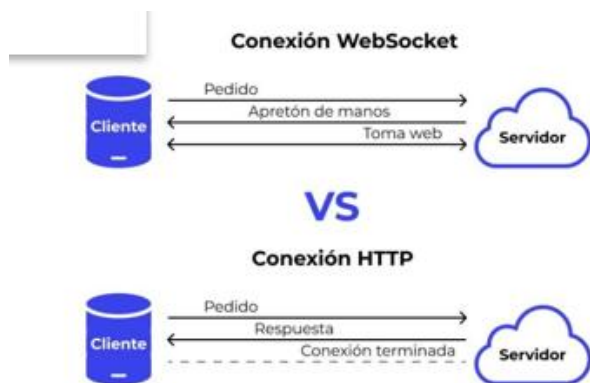


Figura 12. WebSocket

Finalmente, se realizaron pruebas de carga y tolerancia a fallos, comprobando que el sistema mantiene estabilidad ante múltiples operaciones y maneja adecuadamente errores de conexión, tal como se representa en la **Figura 14**.

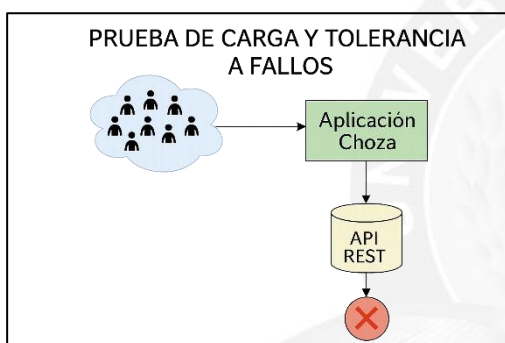


Figura 13. Pruebas

VIII. CONCLUSIONES

- El desarrollo del sistema ChozaPOS permitió implementar una solución tecnológica integral para la gestión de pedidos, mesas y pagos en un entorno de restaurante, utilizando tecnologías modernas como .NET MAUI, Spring Boot y PostgreSQL. La arquitectura basada en servicios web permitió una adecuada separación de responsabilidades, facilitando la comunicación entre la aplicación móvil y el backend mediante el uso de API REST y protocolos de intercambio de datos en formato JSON.
- Asimismo, la integración de mecanismos de comunicación en tiempo real mediante WebSocket permitió mejorar significativamente la sincronización de información entre los distintos módulos del sistema, garantizando una experiencia eficiente en la operación interna. La incorporación

de servicios externos como Dropbox para el almacenamiento de comprobantes fortalece la trazabilidad y el respaldo de las transacciones realizadas.

- Las pruebas realizadas en un entorno controlado demostraron que el sistema es funcional, estable y capaz de manejar múltiples operaciones concurrentes, manteniendo la integridad de los datos y un adecuado manejo de errores. En este sentido, se concluye que Choza POS cumple con los objetivos planteados, proporcionando una herramienta eficiente, escalable y adaptable a las necesidades de gestión de un restaurante.

IX. REFERENCIAS

- [1] Microsoft, “.NET Multi-platform App UI (.NET MAUI) documentation,” Microsoft Docs, 2024. [Online]. Available: <https://learn.microsoft.com/dotnet/maui>.
- [2] VMware, “Spring Boot Reference Documentation,” Spring, 2024. [Online]. Available: <https://spring.io/projects/spring-boot>.
- [3] PostgreSQL Global Development Group, “PostgreSQL Documentation,” 2024. [Online]. Available: <https://www.postgresql.org/docs/>
- [4] R. Richards, “Pro .NET MAUI: Cross-Platform Application Development,” Apress, 2023.
- [5] C. Walls, “Spring in Action,” 6th ed., Manning Publications, 2022.
- [6] M. Fowler, “Patterns of Enterprise Application Architecture,” Addison-Wesley, 2003.
- [7] Postman Inc., “Postman API Platform Documentation,” 2024. [Online]. Available: <https://www.postman.com/>
- [8] Dropbox Inc., “Dropbox API Documentation,” 2024. [Online]. Available: <https://www.dropbox.com/developers>
- [9] J. Bloch, “Effective Java,” 3rd ed., Addison-Wesley, 2018.
- [10] Oracle, “Java Platform Standard Edition Documentation,” 2024. [Online]. Available: <https://docs.oracle.com/en/java/>